The Impact of Action Masking and Environmental Constraints on Rule Internalization in Transformer-Based Chess Agents

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.

In the development of neural agents for combinatorial games, a fundamental tension exists between explicit policy pruning and emergent rule internalization. This thesis investigates two primary paradigms: **Logit Masking** and **Punishment Propagation**.

- **Hypothesis:** While logit masking provides superior convergence speed, unmasked training (Punishment Propagation) forces the network to "grok" the underlying physical mechanics of the environment, potentially yielding more robust latent representations.

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **14**
1.1 Project Background 14
1.2 Project Description 14
1.2.1 Scope and Limitations 15
1.3 Aims and Objectives 15
1.4 Thesis Structure 15

2 Literature Review **16**
2.1 Chess as an MDP 16
2.2 Chess Played by Machines 16
2.2.1 AlphaZero and MuZero 16
2.2.2 Transformers in Chess 17
2.3 Invalid Action Masking 17
2.4 Learning Dynamics 18
2.5 Summary and Research Gap 18

3 System Design and Methodology **19**
3.1 System Architecture Overview 19
3.2 Functional and Non-Functional Requirements 19
3.3 Environment Formulation and State Representation 20
3.3.1 Dual Ruleset MDPs 20
3.3.2 State Canonicalization and Tensor Formulation 20
3.3.3 Network Topology 21
3.3.4 Output Heads 21
3.4 Bipartite Monte Carlo Tree Search 21
3.4.1 Topology 21
3.4.2 Selection and PUCT 21
3.4.3 Exploration Mechanisms 21
3.4.4 Simulation Integrity and the Masking Boundary 22
3.5 Reinforcement Learning Pipeline 22
3.5.1 Self-Play and Replay Buffer 22
3.6 Optimization and the Self-Annealing Loss 22
3.7 Experimental Design and Evaluation Metrics 23
3.7.1 The Configuration Matrix 23
3.7.2 Quantitative Metrics 23

3.8 Discussion of Design Challenges . . . 23

4 Implementation Details . . . 24

4.1 Software Stack and Overview . . . 24
4.2 Chess Engine . . . 25
4.2.1 Bitboards . . . 25
4.2.2 Move Generation . . . 26
4.2.3 Zobrist Hashing . . . 26
4.3 Bipartite Monte Carlo Tree Search . . . 26
4.3.1 Arena Allocators . . . 27
4.3.2 Node Expansion, Promotions, and Terminals . . . 27
4.3.3 Edge Selection and Value Propagation . . . 28
4.3.4 Mask Integration and Prior Renormalization . . . 29
4.4 Neural Network Architecture . . . 29
4.4.1 Tensor Construction . . . 29
4.4.2 Encoder and Output Heads . . . 29
4.5 Training Pipeline . . . 29
4.5.1 Pretraining on Mate Evaluations . . . 29
4.5.2 Batch Expansion and Synchronization . . . 30
4.5.3 The Self-Annealing Loss in Code . . . 30
4.6 Performance and Correctness . . . 31
4.6.1 Throughput and Memory . . . 31
4.6.2 Verification and Testing . . . 31
4.6.3 Reproducibility . . . 31

5 Results and Evaluation . . . 33

5.1 Experimental Setup . . . 33
5.2 Ruleset Convergence Analysis . . . 33
5.3 Masking and Grokking Analysis . . . 33

6 Conclusion . . . 34

6.1 Project Evaluation . . . 34
6.2 Limitations and Future Work . . . 34

7 Statement of Ethics . . . 35

Bibliography . . . 36

List of Figures

List of Tables

Table 1 Hyperparameters held constant across all four configurations. . . . 32

Glossary

$Q(s, a)$	The action-value function representing the mean expected reward for taking action a in state s.
$\pi(a\|s)$	The policy distribution; the probability of selecting action a given the current state s.
$V(s)$	The value function representing the scalar evaluation of a board position's win probability.
λ	The curriculum coefficient (illegal mass ratio) used to weigh policy vs. value loss.
$N(s, a)$	The visit count of an edge in the search tree, indicating how many times an action has been explored.
c_{puct}	A hyperparameter controlling the trade-off between exploration and exploitation in the search tree.
$A_{L(s)}$	The set of strictly legal moves available in state s according to FIDE rules.
$A_{P(s)}$	The set of pseudo-legal moves (geometrically valid but potentially leaving the king in check).
N_{select}	A node in the bipartite MCTS representing a board state where the agent must select a 'from' square.
N_{move}	A node in the bipartite MCTS where an origin square has been fixed and the agent selects a 'to' square.
K_{cap}	The terminal event of king capture used as the reward signal in the pseudo-legal rule set.
d_{model}	The hidden dimensionality of the Transformer encoder's internal representations.

Abbreviations

MCTS	Monte Carlo Tree Search
PUCT	Predictor + Upper Confidence Bound applied to Trees
RL	Reinforcement Learning
MDP	Markov Decision Process
FIDE	Fédération Internationale des Échecs (International Chess Federation)
CNN	Convolutional Neural Network
TD	Temporal Difference (Learning)
FIFO	First-In, First-Out (Replay Buffer)
SOTA	State Of The Art
MSB / LSB	Most Significant Bit / Least Significant Bit
W/D/L	Win / Draw / Loss
FEN	Forsyth-Edwards Notation

Chapter 1

Introduction

1.1 Project Background

Modern machine learning frequently decouples models from environment transition dynamics via **Logit Masking**, a heuristic that artificially zeroes out the probability of illegal actions prior to softmax activation. While computationally efficient, this deprives the network of negative feedback for invalid predictions, bypassing the model's need to internally represent the mechanics of play. Furthermore, standard reinforcement learning (RL) agents are prevented from making invalid state transitions by hardcoded environmental constraints.

This raises a question that the literature has largely sidestepped: when these guardrails are removed, what learning dynamics emerge in their absence, and how are they reflected in the network's internal representations? Chess provides an optimal substrate for this investigation: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space.

1.2 Project Description

Existing literature rarely assesses the impact of these environmental guardrails on the learning dynamics of transformer-based agents. This project investigates two orthogonal axes of environmental constraint: Action Space Constraints and Logit Masking.

To facilitate explicit measurement of model interpretability, a custom two-pass autoregressive transformer encoder and a bipartite Monte Carlo Tree Search (MCTS) are introduced. This architecture decouples piece selection from destination selection, providing a granular window into the model's spatial reasoning.

The two experimental axes give rise to four configurations:

1. Along the first axis, the control uses a strict legal ruleset, while the test condition adopts a pseudo-legal ruleset whose terminal state is the dense king-capture event K_{cap} rather than the sparse, recursive checkmate. This shift alters the MDP reward function $R(s, a)$, yielding a potentially smoother reward landscape. The hypothesis is that the network will first bootstrap the rules and goals of the environment; ideal play in the pseudo-legal space A_P may converge to standard chess, with concepts such as pins and checkmates emerging as survival heuristics to prevent or secure king capture.

Conversely, removing environmental guardrails may degrade MCTS value estimation and prevent policy bootstrapping.

2. Along the second axis, the control applies logit masking (illegal moves are eliminated before selection), while the test configuration leaves logits unmasked, so the network is permitted to propose illegal moves and is penalised through the loss function. The hypothesis is that unmasked training will incur an initial sample-efficiency penalty by forcing a dual-optimization problem: learning game mechanics alongside strategy, but may ultimately yield more robust internal representations and stronger asymptotic play.

1.2.1 Scope and Limitations

Three limitations bound the scope of the work. First, playing strength is not benchmarked against external engines; the comparison of interest is across the four configurations. Absolute Elo against SOTA would require orders of magnitude more compute than the project budget permits. Second, the value head is bootstrapped against MCTS root values rather than terminal outcomes, constrained by single GPU throughput. Third, the four configurations are trained to a fixed wall-clock budget rather than to convergence, investigating learning trajectories over asymptotic performance.

1.3 Aims and Objectives

1. The primary aim is to train multiple transformer-based agents via self-play reinforcement learning across the varying rule sets and action spaces and to quantitatively evaluate their performance, learning trajectories, and emergent behaviours. Concrete objectives include implementing a highly optimised, dual-ruleset chess library for high-throughput RL self-play; architecting a two-pass autoregressive transformer and bipartite MCTS that models piece-then-square action selection; training the four configurations $(\text{Legal} + \text{Masked}, \text{Legal} + \text{Unmasked}, \text{Pseudo-Legal} + \text{Masked}, \text{Pseudo-Legal} + \text{Unmasked})$ from scratch under identical hyperparameters; quantifying convergence rates, illegal-move proposal frequencies, and the emergence of rule-abiding play across the unconstrained setups; and correlating the emergence of legal play with attention distributions in the two-pass encoder, thereby opening a window onto spatial reasoning.

1.4 Thesis Structure

The remainder of this dissertation is structured as follows. **Chapter 2** surveys the literature surrounding reinforcement learning, Transformer architectures, action masking, and curriculum learning, identifying the gap this work addresses. **Chapter 3** details the system design, including the bipartite MCTS, the two-pass encoder, and the self-annealing loss function. **Chapter 4** outlines the implementation of the Rust engine and training pipeline, with particular attention to the engineering decisions that make high-throughput self-play tractable. **Chapter 5** presents the results and evaluation across the four configurations. **Chapter 6** concludes and discusses avenues for future work.

Chapter 2

Literature Review

2.1 Chess as an MDP

Chess is a deterministic, zero-sum, perfect-information game, formally modelled as a Markov Decision Process defined by the tuple (S, A, T, R), where S is the state space ($|S| \approx 10^{40}$), A is the action space, $T : S \times A \to S$ is the deterministic transition function, and $R : S \times A \to \{-1, 0, 1\}$ is the reward function (Shannon, 1950). Standard implementations enforce a legal action space $A_{L(s)} \subset A$ in which moves leaving the king in check are excluded by the environment (World Chess Federation (FIDE), 2023). This project introduces a relaxed pseudo-legal action space $A_{P(s)}$ such that $A_{L(s)} \subset A_{P(s)} \subset A$, permitting all geometrically valid piece movements and shifting the terminal condition from the abstract *checkmate* to the explicit *king capture* K_{cap}.

Typical chess agents emit a single joint policy distribution over the ≈ 4672 possible piece movement on all 64 squares. Factoring the joint probability as

$$P(a|s) = P(\text{from}|s) \cdot P(\text{to}|s, \text{from})$$

collapses the output dimensionality from $d_{\text{model}} \times 4672$ to $d_{\text{model}} \times 64$ and gives rise to two distinct, observable failure modes.

For any state s, an optimal policy π^* trained in A_P must implicitly learn to prune the action space such that:

$$\pi^*(a|s) \to 0 \quad \text{for all } a \in \left(A_{P(s)} \setminus A_{L(s)}\right)$$

Under this paradigm, concepts such as pins and checkmating nets cease to be hardcoded environmental properties and become *emergent survival heuristics* within the value landscape.

2.2 Chess Played by Machines

2.2.1 AlphaZero and MuZero

The paradigm of self-play RL in chess was defined by AlphaZero (Silver *et al.*, 2017), which combined Monte Carlo Tree Search (MCTS) with deep convolutional neural networks (CNNs) for policy and value evaluation. However, AlphaZero relies entirely on a hardcoded

environment to generate $A_{L(s)}$, bypassing the need for the network to internalize the rules of the game.

MuZero (Schrittwieser *et al.*, 2020) removed the need for a known transition dynamics model by learning a latent environment simulator. While MuZero demonstrates the ability to adhere to environment constraints implicitly within its hidden state transitions, its policy head still simulates within the legal action space during MCTS rollouts. The literature lacks insight into how the removal of environmental guardrails or action masking impacts the sample efficiency and representational robustness of the underlying network.

2.2.2 Transformers in Chess

Recent advancements demonstrate the efficacy of Transformer architectures in chess, shifting away from the spatial inductive biases of CNNs. (Monroe and Chalmers, 2024) achieved Grandmaster-level performance with a Transformer relying purely on attention to evaluate board states, without explicit search. (Jenner *et al.*, 2024) provided complementary evidence of learned look-ahead in chess-playing networks, showing that attention heads encode candidate future board states. (McGrath *et al.*, 2022) further demonstrated that AlphaZero-style networks acquire human-interpretable concepts (material balance, mobility, king safety) over the course of training.

These results frame the central architectural choice of the present work. Decoupling $P(\text{from}|s)$ from $P(\text{to}|s, \text{from})$ forces attention to explicitly represent geometric constraints in two separable phases, providing an observable window into distinct failure modes: attention failure (identifying movable pieces) and kinematic failure (how pieces move).

2.3 Invalid Action Masking

In policy gradient algorithms, it is standard practice to prevent the selection of illegal actions via Invalid Action Masking (Logit Masking). Masking applies a transformation to the policy logits z prior to the softmax activation:

$$P(a_i|s) = \frac{\exp(z_i + M_i)}{\sum_j \exp(z_j + M_j)}$$

where $M_i = -\infty$ if $a_i \notin A_{L(s)}$, and 0 otherwise.

(Huang and Ontañón, 2020) demonstrated that masking significantly improves sample efficiency by collapsing the exploration space. However, logit masking, by design, prevents the network receiving feedback from masked indices due to gradients being nullified in the compute graph preventing backpropagation of loss. For an MCTS-based agent, masking is essentially required as simulations into invalid state transitions corrupt value estimates and propagate poison up the tree. However, Huang and Ontañón's analysis is restricted to small, flat action spaces where the legal subset is a fixed function of a few features (MicroRTS). Chess presents a qualitatively harder case: legality depends on the global board configuration, and the function $A_{L(s)}$ is itself nontrivial to compute. Whether the conclusions of (Huang and Ontañón, 2020) generalize to a setting where the rules themselves constitute an algorithmic learning target aims to be answered.

2.4 Learning Dynamics

The unmasked, pseudo-legal regime forces the network to learn the rules of chess from scratch as a precondition for strategic evaluation, raising the prospect of non-monotonic learning dynamics. (Power *et al.*, 2022) identified the phenomenon of *grokking*, in which neural networks trained on small algorithmic datasets exhibit sharply delayed generalization long after overfitting the training set, mediated by weight-decay pressure. Chess is an algorithmic dataset embedded in a strategic MDP, showcasing the potential of analogous phase transitions.

(Bengio *et al.*, 2009) formalized *curriculum learning*, showing that ordering training data from simple to complex acts as a regularizer, guiding optimization toward superior local minima inaccessible through random shuffling. It is hypothesized a pseudo-legal environment serves implicitly as curriculum where an agent first receives rewards through piece kinematics and king capture before survival heuristics under A_L.

(Sharma *et al.*, 2017) introduced a framework for factoring discrete action spaces into compositional, basis-like components, enabling cross-action generalization among actions that share common factors. This principle is instantiated in the factored engine architecture, where each selection and destination shares a common output head.

2.5 Summary and Research Gap

While the efficacy of action masking is well-documented in general RL ((Huang and Ontañón, 2020)), and Transformers have proven capable of modelling chess at a high level ((Jenner *et al.*, 2024)), the intersection of these domains is unexplored. No existing work isolates the impact of environmental guardrails on the representational quality of Transformer-based MCTS agents, nor on the dynamics of rule internalization when those guardrails are removed.

In the unmasked, pseudo-legal regime, the network faces a dual-optimization problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). It is hypothesized that this configuration will exhibit a grokking-like phase transition—an initial random-walk regime of high illegal-move proposal, a sharp convergence toward $A_{P(s)}$ as kinematics are internalized, and only then the development of strategic play in $A_{L(s)}$.

This project addresses the gap by training the four configurations from scratch under identical hyperparameters, instrumenting them with a factored two-pass encoder, and providing a quantitative analysis of how rule internalisation and action-space pruning manifest within the attention layers of a chess-playing agent.

Chapter 3

System Design and Methodology

3.1 System Architecture Overview

The system is designed as a closed-loop reinforcement learning pipeline, decoupling the environment transition dynamics from the neural network's internal representations. The architecture consists of three primary components: a custom deterministic environment simulator supporting dual rulesets, a two-pass autoregressive Transformer encoder, and a bipartite Monte Carlo Tree Search (MCTS) algorithm. The system operates via self-play, generating trajectories that are aggregated into a global FIFO replay buffer to optimize the network via gradient descent.

Each component is designed so that the experimental axes—rule set and masking—can be toggled independently without altering any other part of the system. This is essential to the validity of the comparison: the four configurations differ only in the two boolean flags governing their respective axes, with all hyperparameters held constant.

3.2 Functional and Non-Functional Requirements

The methodology imposes four hard requirements on the implementation, each of which constrains the design space in Section 3.3–Section 3.7.

1. **Configuration parity:** The four cells of the experimental matrix must differ only in the two boolean flags governing rule set and masking. No code path may exist that is reachable in one configuration but not another, with the sole exception of the masking step itself. This rules out per-configuration training scripts and motivates the unified `forward_classification` implementation in Section 4.4.2.
2. **Self-play throughput:** Four full training runs under a single-GPU budget require sustained throughput of order 10^4 position evaluations per second. This rules out reference-counted tree structures (cache-hostile) and dynamic move-list allocation (heap pressure), motivating arena allocation (Section 4.3.1) and stack-allocated move lists (Section 4.2.2).

3. **Reproducibility:** Self-play is intrinsically stochastic, but every source of randomness must be seedable. Dirichlet noise, temperature sampling, replay sampling, and weight initialization all consume from seeded RNGs.

4. **Auditability:** Observed learning dynamics must be attributable to the network rather than to engine artefacts. This requires the move generator to be independently testable against a reference (Section 4.6.2) and the masking boundary in Section 4.2.2 to be unambiguous in code.

3.3 Environment Formulation and State Representation

3.3.1 Dual Ruleset MDPs

The **Legal Environment** $\boldsymbol{E_l}$ adheres to standard FIDE rules. The action space ($A_{l(s)}$) is strictly contained to moves that do not leave the king in check. The terminal reward $R(s, a)$ is evaluated upon Checkmate, Stalemate, or standard draw conditions. The **Pseudo-Legal Environment** E_P relaxes the action space to $A_{P(s)}$, permitting all geometrically valid piece movements regardless of king safety. The terminal condition shifts from checkmate to the explicit king-capture event K_{cap}. FIDE draw conditions are retained to guarantee finite rollouts and to allow a correct transition into legal play.

3.3.2 State Canonicalization and Tensor Formulation

To enforce symmetric learning and halve the state space, all board states are canonicalized to the perspective of the side-to-move. If it is Black's turn, the board, and castling rights are geometrically flipped. The state s is mapped to the spatial tensor representation $X \in \mathbb{R}^{64\times 14}$ and a scalar meta-tensor $M \in \mathbb{R}^5$.

- **Spatial Tensor**: Comprises 12 piece bitboards (6 piece types $\times$ 2 colors), 1 en-passant target plane, and 1 selected-square plane (utilized in the second pass of the network).
- **Meta Tensor**: Encodes the 4 castling rights (one-hot) and the half-move clock (scaled by $\frac{1}{100}$).

To explicitly measure spatial reasoning, the system factors the joint probability of a move $P(a|s)$ into an autoregressive sequence: $P(a|s) = P(\text{from}|s) \times P(\text{to}|s, \text{from})$. The network executes two forward passes using shared weights:

1. **Pass 1 (Piece Selection)**: The network evaluates the board state with the selected square plane zeroed out, outputting a policy distribution over the 64 squares to select the origin square (*from*). The value head will evaluate the board state.
2. **Pass 2 (Destination Selection)**: The origin square is encoded into the 14th plane of the spatial tensor. The network re-evaluates the state, outputting a policy distribution over the 64 squares to select the destination (*to*). The value head will evaluate the position given a piece to move, effectively evaluating the viability of the selected piece.

Promotions: The factored $64 \to 64$ output lacks a third dimension for promotion piece selection. Thus, promotions are handled in the search topology: when a pawn destination on the back rank is expanded, the corresponding edge is replaced by four sub-edges (Q, R, B, N), equally dividing the prior probability. This delegates under-promotion to MCTS, where the additional branching factor is amortized over many simulations.

3.3.3 Network Topology

The model uses a Transformer Encoder architecture without spatial inductive biases (e.g., convolutions). The spatial tensor X is linearly projected to d_{model}, and $2D$ learned positional embeddings are added to retain geometric context. The meta-tensor M is linearly projected and concatenated as a pseudo-[CLS] token, resulting in a sequence length of 65.

The model is scaled symmetrically: $n_{\text{heads}} = 8$, $n_{\text{layers}} = 8$, $d_{\text{model}} = 8 \cdot 64 = 512$, $d_{\text{ff}} = 4 \cdot d_{\text{model}}$.

3.3.4 Output Heads

The final latent representation is routed into two distinct heads:

1. **Policy head:** a linear projection of the 64 spatial tokens to $\mathbb{R}^{64}$, producing unnormalized logits over square selection.
2. **Value head:** a linear projection of the [CLS] token to $\mathbb{R}^3$, producing W/D/L logits. In Pass 1 this evaluates the raw state; in Pass 2 it evaluates the state conditional on the selected origin square.

3.4 Bipartite Monte Carlo Tree Search

3.4.1 Topology

To accommodate the two-pass autoregressive policy, the search tree alternates between two node types:

- **Selection Nodes (N_{select}):** Represent a board state. Edges represent the choice of an origin square.
- **Action Nodes (N_{move}):** Represent a board state plus a selected origin square. Edges represent the destination square; transitioning the environment to a new N_{select} node.

A complete move thus traverses a (Selection, Action) pair. This bipartite structure is not merely a notational convenience: it physically separates the two failure modes of the factored policy in the search tree, such that visit-count statistics on Selection edges directly diagnose attention failures, while Action edges diagnose kinematic failures.

3.4.2 Selection and PUCT

During traversals, edges are selected using a modified Predictor + Upper Confidence Bound applied to Trees (PUCT) algorithm. To mitigate draw-seeking behaviour, a contempt factor is baked into the empirical action value $Q(s, a)$.

$$
\begin{aligned}
Q(s,a) &= W - L - 0.05D \\
U(s,a) &= c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)} + 10^{-8}}{1 + N(s,a)} \\
\text{PUCT}(s,a) &= Q(s,a) + U(s,a)
\end{aligned}
$$

where W, D, L are the mean W/D/L value estimates accumulated on the edge, $P(s, a)$ is the prior, and $N(s)$, $N(s, a)$ are the parent and edge visit counts respectively.

3.4.3 Exploration Mechanisms

Exploration is injected through two mechanisms. Dirichlet noise is applied exclusively to the root prior, with $P(s, a) \leftarrow (1 - \varepsilon)P(s, a) + \varepsilon \cdot \eta$, with $\eta \sim \text{Dir}(\alpha)$, $\alpha = 0.3$, and

$\varepsilon = 0.25$. At the root, the move played is sampled in proportion to $N(s,a)^{\frac{1}{t}}$, where the temperature t scales dynamically with ply count and remaining material, transitioning from exploratory in the opening to deterministic in endgames.

3.4.4 Simulation Integrity and the Masking Boundary

During MCTS rollouts, invalid transitions corrupt value backups across all ancestor edges. To maintain simulation integrity, action masking is applied during the search phase across all configurations, ensuring the search tree only explores legal moves. The experimental distinction lies in whether action masking is applied to the network's raw logits during loss calculation. In masked configurations, illegal logits are clamped to -10^9 before softmax, training the model against a distribution restricted to legal actions. In unmasked configurations, the policy loss is computed over the raw 64-square distribution; the network must internalize the penalty for assigning probability mass to illegal moves via cross-entropy. While the MCTS always sees a legal distribution, the training signal differs in whether the network is shielded from or penalized for invalid predictions.

3.5 Reinforcement Learning Pipeline

3.5.1 Self-Play and Replay Buffer

Trajectories are generated via highly parallelized self-play. Samples are stored in a FIFO replay buffer of capacity $2^{19} = 524288$. To prevent infinite games, rollouts are forcibly terminated as draws after 400 plies, or whenever the MCTS root value estimate for a draw exceeds 0.95, relaxed to 0.75 after 60 plies.

3.6 Optimization and the Self-Annealing Loss

The network is optimized with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.99$, weight decay 10^{-4}) under a Noam learning-rate schedule with factor 0.01 and 4000 warm-up steps. The value head is bootstrapped against the MCTS root-value distribution rather than the eventual game outcome (TD-like) for computational feasability.

The principal methodological contribution lies in the loss function. For a sample (s, π, z) with policy prediction p and value prediction v, the loss is:

$$L = (1+\lambda) \cdot \underbrace{\left(-\sum_a \pi(a|s) \log p(a|s) \right)}_{L_{\text{policy}}} + (1-\lambda) \cdot \underbrace{\left(-\sum_i z_i \log v_i \right)}_{L_{\text{value}}}$$

where $\lambda \in [0,1]$ is the mean probability mass that the network has assigned to illegal moves over the most recent batch of leaf expansions.

This acts as a strict mathematical curriculum. When the network proposes many illegal moves $\lambda \approx 1$, the policy loss approaches 2 and the value loss approaches 0. The network is forced to learn piece kinematics before state evaluation. As the model internalizes piece kinematics, the weights balance to $1:1$. This approach to curriculum learning is unique to the unmasked configuration, as λ is dependent on illegal move weights (λ is always zero in masked configurations).

3.7 Experimental Design and Evaluation Metrics

3.7.1 The Configuration Matrix

The four configurations form a 2×2 matrix. **Legal + Masked** serves as the control, reproducing the standard AlphaZero paradigm. **Legal + Unmasked** requires the network to learn piece kinematics through loss penalties while the environment continues to enforce legality. **Pseudo-Legal + Masked** lets the network play the K_{cap} variant under environmental masking, so it must discover that kings should not be left in check as a terminal-avoidance heuristic. **Pseudo-Legal + Unmasked** forces the network to learn both piece kinematics and FIDE-style legality entirely from scratch.

3.7.2 Quantitative Metrics

The learning dynamics of each configuration are captured continuously during training by four metrics: **average illegal probability** (the total probability mass assigned to invalid actions, a direct readout of rule internalisation), **average game length** in plies (indicating the transition from random walks to sustained strategic play), **win/draw ratios** (tracking the emergence of decisive outcomes), and **value loss** (measuring the calibration of the network's state evaluation over time).

3.8 Discussion of Design Challenges

Three design problems were materially harder than they first appeared, and each shaped the final system in ways worth recording.

TODO

Chapter 4

Implementation Details

This chapter details the software architecture and engineering paradigms utilized to construct the self-play reinforcement learning system. The implementation translates the methodology into a high-throughput, reproducible execution environment optimized for three criteria: clean experimental toggling, sufficient velocity to complete four training runs within the project timeframe, and system auditability to isolate learning dynamics from engine artifacts.

The architecture comprises a custom bitboard-based chess engine, a bipartite Monte Carlo Tree Search (MCTS) utilizing flat arena allocators, and a shared-weight Transformer encoder implemented via the Burn deep learning framework. All components are parallelized to bridge theoretical design with optimized execution.

4.1 Software Stack and Overview

The system is implemented entirely in Rust, leveraging the language's strict aliasing rules and ownership model to guarantee data-race-free parallelization. Neural network operations are executed via the Burn framework, which provides a backend-agnostic tensor computation graph (WGPU for portable execution, CUDA for hardware acceleration).

The self-play pipeline (Algorithm 1) is designed for maximum throughput via lock-free concurrency. At the initialization of each training iteration, B parallel game instances are spawned. Because each MCTS instance encapsulates its own memory arenas, the Rayon library's `par_iter_mut()` is utilized to distribute tree traversals, leaf expansions, and backpropagations across all available CPU cores without mutex contention. Synchronization only occurs at the neural network interface, where leaf nodes from all B trees are aggregated into a single contiguous batch for GPU inference.

Reinforcement Learning Loop

```
Require: Model weights θ, Replay Buffer D (capacity 2^19), Batch size B
Initialise B independent MCTS instances {T_1, ..., T_B}
while training do
    for step = 1 to steps_per_iteration do
        for i = 1 to simulations_per_move do
            parallel for each tree T in {T_1, ..., T_B} do
                Traverse T to find leaf node n
            end
            Synchronise threads
            Batch inputs from all leaves and execute forward pass(θ)
            parallel for each tree T do
                Expand leaf n using network outputs
                Backpropagate value estimates to root
            end
        end
        parallel for each tree T do
            Select action a ~ π(a|s) based on visit counts
            Store transition (s, π, z) in D
            Advance environment state
            if terminal state reached then reset T
        end
    end
    if |D| > min_samples then
        for epoch = 1 to gradient_steps do
            Sample mini-batch from D
            Compute composite loss L(θ)
            Update θ via AdamW
        end
    end
end
```

4.2 Chess Engine

The environment is a custom chess engine optimized specifically for MCTS rollouts. State representation is strictly packed, and move generation relies on compile-time precomputed masks to minimize branching and minimize CPU cache misses.

4.2.1 Bitboards

The board state is encoded within the `ChessBoard` struct using a `[[Bitboard; 6]; 2]` array, representing the 6 piece types across 2 colors, alongside 3 aggregate occupancy bitboards (White, Black, All). The struct is annotated `#[repr(align(64))]` so that the entire state

fits within two CPU cache lines, making move generation cache-friendly under aggressive parallel access. Hardware intrinsics, specifically `trailing_zeros` and `leading_zeros`, are used for $O(1)$ piece iteration and bit isolation.

4.2.2 Move Generation

The engine employs a two-step deferred verification architecture for move generation: **Precomputed Lookup Tables:** Knight, king, pawn, rook, bishop, and rook/bishop direction masks, together with a 64×64 `BETWEEN` table, are constructed via `const fn` and embedded directly in the binary as `pub const` arrays. This elides the static initialization phase entirely and allows the compiler to fold lookups into immediate operands where it can prove the index.

Branchless Sliding-Piece Generation: Sliding pieces are generated by AND-ing each direction mask with the global occupancy and isolating the first blocker via LSB or MSB depending on the direction:

```
let to_sq = if i < 2 { blockers.pop_msb() } else { blockers.pop_lsb() };
```

The blocker square is included in the ray if it is occupied by an enemy piece (a capture), and the precomputed `BETWEEN[from][blocker]` mask supplies the squares strictly between origin and blocker, to be trivially iterated over to generate sliding moves.

Stack-Allocated Move Lists: Generated moves are written to a stack-allocated `ArrayVec<ChessMove, 128>`, safely bounding chess's empirical maximum branching factor while eliminating heap allocation overhead.

Deferred Legality Verification: Rather than implementing computationally expensive pin-detection logic during generation, legality is verified post-hoc. The `is_legal()` method clones the board state, applies the pseudo-legal move, and evaluates if the resulting state leaves the active King under attack via `is_square_attacked()`. This deferred approach significantly reduces branching complexity while maintaining high throughput.

4.2.3 Zobrist Hashing

State repetition detection is implemented via Zobrist hashing. A static table of 64-bit pseudo-random keys (`ZobristKeys`) is lazily initialized using Rust's `OnceLock`. The hash is computed in $O(P)$ time, where P is the number of active pieces, by XOR-ing the keys corresponding to the current board state, castling rights, en-passant file, and side-to-move. While incremental $O(1)$ hashing is theoretically optimal for sequential play, the $O(P)$ approach was selected to eliminate complex state-tracking across independent MCTS nodes. Profiling indicates this computational cost is negligible relative to network inference.

4.3 Bipartite Monte Carlo Tree Search

A standard MCTS represents a complete action as a single edge between two states. To map the search topology onto the network's two-pass autoregressive policy, the search tree is implemented as a bipartite graph alternating `PieceSelect` and `PieceMove` nodes (Section 3.4).

4.3.1 Arena Allocators

To avoid the memory fragmentation and pointer-chasing overhead associated with recursive tree structures, the MCTS utilizes flat, index-based arena allocators:

```
struct Arena<T> {
    buffer:   Vec<T>,
    freelist: Vec<usize>,
}
```

- `node_arena: Arena<MctsNode>`
- `edge_arena: Arena<MctsEdge>`
- `position_arena: Arena<ChessPosition>`

Nodes reference child edges and associated board states via `usize` indices. A dedicated `position_arena` is implemented to prevent deduplication of board state over consecutive `PieceSelect` and `PieceMove` nodes.

To address memory pressure, arenas are backed by freelists. When subtrees are abandoned (e.g. after a move is chosen), their indices are returned to the freelist. Bulk edge insertion (`push_block`) searches for contiguous free slots. A garbage-collection pass recursively frees all nodes, edges, and board positions of abandoned subtrees asynchronously behind gradient steps. This keeps RAM proportional to the active search tree, enabling larger batch sizes.

4.3.2 Node Expansion, Promotions, and Terminals

Leaf expansion (Algorithm 2) transitions the state machine. Expanding a `PieceSelect` edge generates a `PieceMove` node without altering the underlying board state. Expanding a `PieceMove` edge applies the move, computes the new Zobrist hash, evaluates terminal states (including threefold repetition via path history), and generates a new `PieceSelect` node.

Promotions are handled natively by the bipartite structure. When a pawn-to-back-rank destination is expanded, the MCTS intercepts the spatial prior p and distributes it uniformly ($p/4$) across four discrete edges (Q, R, B, N), each carrying the corresponding promotion. This enables the model to handle underpromotions through search topology rather than increased output dimensionality.

Bipartite Leaf Expansion

```
Require: Leaf edge e, Network prior probabilities P, Search path history H
Let N_parent be the parent node of e
if N_parent is of type PieceSelect then
    Create N_child of type PieceMove
    Set N_child.selected_square = e.target_square
    Link e to N_child
else if N_parent is of type PieceMove then
    s' ← Apply move(N_parent.selected_square, e.target_square)
    if Zobrist(s') appears ≥ 2 times in H then
        Create N_child as Terminal(Draw)
    else if is_terminal(s') then
        Create N_child as Terminal(Win/Loss)
    else
        Create N_child of type PieceSelect representing s'
    end
    Link e to N_child
end
if N_child is not Terminal then
    for each legal destination square i do
        if move is pawn promotion then
            Create 4 edges (Q, R, B, N) with prior = P[i]/4
        else
            Create 1 edge with prior = P[i]
        end
        Attach edge(s) to N_child
    end
end
```

The fifty-move rule has been shortened to forty full moves (eighty plies) to accelerate training.

4.3.3 Edge Selection and Value Propagation

PUCT scoring uses the contempt-augmented variant derived in Section 3.4.2:

- $Q_e = W_e - L_e - 0.05D_e$
- $U_e = c_{\text{puct}} \cdot p_e \cdot \frac{\sqrt{N}+10^{-8}}{1+n_e}$

The selected edge maximizes $Q_e + U_e$. On leaf evaluation, the value vector $[W, D, L]$ is back-propagated up the path. The vector is inverted ($[W, D, L] \leftarrow [L, D, W]$) if the current side-to-move $\neq$ leaf side-to-move. The path is cleared after each simulation for the next traversal.

4.3.4 Mask Integration and Prior Renormalization

The network outputs a probability distribution over all 64 squares in unmasked configurations, as opposed to just the valid squares when masked. To produce a valid prior for PUCT, the MCTS computes the illegal mass $\lambda = \sum_{i \notin A_{L(s)}} p_i$ and renormalizes the legal priors:

$$p'_i = \frac{p_i}{1 - \lambda} \quad \forall i \in A_{L(s)}$$

The same λ is the quantity surfaced to the loss function as the curriculum coefficient described in Section 4.5.3.

4.4 Neural Network Architecture

The `ChessTransformer` is a shared-weight encoder mapping the spatial tensor and the meta-vector to a policy distribution and a W/D/L estimate.

4.4.1 Tensor Construction

Inputs are first canonicalized to the side-to-move via `flip_board()`, then written into flat `Vec<f32>` slices before being reshaped through Burn's `TensorData` API. This guarantees a contiguous memory layout before the host-to-device transfer, avoiding allocation churn.

- **Spatial tensor ($\boldsymbol{X} \in \mathbb{R}^{64\times 14}$):** 12 piece planes, 1 en-passant plane, 1 selected-square plane (zero on the first pass, the highlighted origin square on the second).
- **Meta-tensor ($\boldsymbol{M} \in \mathbb{R}^{5}$):** 4 binary castling rights and 1 continuous half-move clock.

4.4.2 Encoder and Output Heads

The 64 spatial vectors are linearly projected to $d_{\text{model}} = 512$, and learnable rank/file embeddings of size 8 each are summed and broadcast across the grid before being added to the spatial token sequence. The meta-vector is projected and concatenated as a pseudo-[CLS] token at index 64, yielding a $65 \times d_{\text{model}}$ sequence consumed by the encoder ($n_{\text{layers}} = 8$, $n_{\text{heads}} = 8$).

After the encoder, the sequence bifurcates:

- Tokens 0..63 pass through a linear policy head producing 64 spatial logits.
- Token 64 is passed through a linear value head producing the W/D/L logits.

The model's `forward_classification` method dynamically applies action masking based on configuration. In masked configurations, a boolean tensor identifies illegal squares and `mask_fill` clamps the corresponding logits to -10^9 before softmax. In unmasked configurations this step is bypassed and the cross-entropy is computed against the raw distribution, enabling the creation of the curriculum coefficient λ in Section 4.5.3.

4.5 Training Pipeline

The training pipeline orchestrates data generation, buffer management, and gradient updates, with each stage engineered to minimize idle time on either CPU or GPU.

4.5.1 Pretraining on Mate Evaluations

Before the self-play loop begins, the value head is bootstrapped against a small dataset of positions annotated with mate-in-N evaluations (`mate_evals.tsv`). One hundred gradient

steps are run with the loss ratio fixed at -1 to ignore policy output. This step is negligible in wall-clock cost and greatly improves search tree efficiency.

4.5.2 Batch Expansion and Synchronization

The interface between the CPU-bound MCTS and the GPU-bound inference is implemented in `expand_batch` (Algorithm 3). MCTS instances traverse independently in parallel until each has selected a leaf. The threads then synchronize, aggregating canonicalized input tensors and masks into a single batch. Following a unified forward pass, the resulting `NetworkLabels` (policy and value arrays) are scattered back to their respective MCTS instances for edge expansion and value backpropagation. This single-batch boundary is the only synchronization in the entire self-play loop, scaling compute cost with batch size so long as the GPU has memory to spare.

Batched Inference and Prior Renormalization

Require: Set of leaf nodes $\{n_1, ..., n_B\}$, Configuration C

$X_{\text{batch}}, M_{\text{meta}} \leftarrow$ Extract spatial and meta tensors from $\{n_1, ..., n_B\}$
$M_{\text{mask}} \leftarrow$ Generate legal move boolean masks for all $b \in B$
if C.masked is True **then**
  $P_{\text{raw}}, V_{\text{raw}} \leftarrow \text{TransformerForward}(X_{\text{batch}}, M_{\text{meta}}, M_{\text{mask}})$
else
  $P_{\text{raw}}, V_{\text{raw}} \leftarrow \text{TransformerForward}(X_{\text{batch}}, M_{\text{meta}}, \text{None})$
end
for $b = 1$ **to** B **do**
  $\lambda \leftarrow 0$
  if C.masked is False **then**
    for each illegal square i in n_b **do**
      $\lambda \leftarrow \lambda + P_{\text{raw}[b]}[i]$
    end
    for each legal square j in n_b **do**
      $P_{\text{raw}[b]}[j] \leftarrow P_{\text{raw}[b]} \frac{[j]}{1-\lambda}$
    end
  end
  n_b.illegal_mass $\leftarrow \lambda$
  n_b.value $\leftarrow V_{\text{raw}[b]}$
  Populate n_b edges using $P_{\text{raw}[b]}$
end

4.5.3 The Self-Annealing Loss in Code

The composite loss of Section 3.6 is computed in `calculate_loss`, which is structurally identical across all four configurations, where only the value of λ (here 'ratio') varies:

```
let policy_probs = log_softmax(policy_pred, 1);
let policy_loss = (target_policy * policy_probs).sum_dim(1).mean().neg();
```

```
let value_probs = log_softmax(value_pred, 1);
let value_loss = (target_value * value_probs).sum_dim(1).mean().neg();

(policy_loss * (1.0 + ratio)) + (value_loss * (1.0 - ratio))
```

The `log_softmax`-then-multiply formulation is mathematically equivalent to a softmax followed by KL divergence against soft targets, but avoids the numerical instability of computing the softmax explicitly when logits span a wide range. In the masked configurations, `ratio` is naturally 0 and the loss reduces to the standard, equally weighted policy/value sum, recovering AlphaZero's objective without a separate code path.

4.6 Performance and Correctness

4.6.1 Throughput and Memory

Engine throughput is dominated by the batched forward pass of the Transformer, with tree search and arena garbage collection running concurrently behind it. At a batch size and simulation count of 256 each, steady-state resident memory sits at approximately 11 GB system RAM and 10 GB VRAM. CPU usage hovers at 7% while the GPU (NVIDIA RTX 4000 Ada) maxes out at 100%, indicating inference to be the major bottleneck.

4.6.2 Verification and Testing

Three independent layers of verification target the three independent failure modes of the system.

Move generation is tested against the standard perft position suite (Kiwipete, Position 3, Position 4, Position 5, plus the initial position), exercising en-passant, castling through check, promotion, and double-pawn-push edge cases.

State machine integrity is checked by round-tripping FEN: every position reached during a 1000-game self-play smoke test is exported to FEN, parsed back, and compared for bitwise equality. Zobrist hashes are verified to be invariant under FEN round-trips and to match across symmetric-castling-rights configurations.

Network–engine boundary. Tensor shapes are verified at compile time through Burn's typed tensor API, eliminating tensor mismatch bugs. Zobrist hashes use 64-bit keys, providing cryptographic-grade collision resistance over the search depths encountered during training.

4.6.3 Reproducibility

All hyperparameters used across the four training runs are summarized in Table 1. The same values are used for every cell of the experimental matrix; the only variables are the two boolean flags `legal` and `masked`.

Parameter	**Value**
Batch size	256
MCTS simulations / move	256
Replay buffer capacity	2^{19}
Gradient steps / iteration	256
Self-play steps / iteration	256
d_{model}	512
n_{layers}	8
n_{heads}	8
d_{ff}	2048
c_{puct}	1.25
Dirichlet α	0.3
Dirichlet ε	0.25
Contempt (draw penalty)	0.05
Optimiser	AdamW
β_1, β_2	0.9, 0.99
Weight decay	10^{-4}
LR schedule	Noam, factor 0.01, 4000 warmup
Max ply / game	400
Half-move limit	80 plies
Pretraining steps (mate eval)	100
Random seed	1234

Table 1: Hyperparameters held constant across all four configurations.

Chapter 5

Results and Evaluation

5.1 Experimental Setup

5.2 Ruleset Convergence Analysis

5.3 Masking and Grokking Analysis

Chapter 6

Conclusion

6.1 Project Evaluation

6.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of deep late-stage grokking.
- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Kinematic Bootstrapping:** Boot strap the policy head with uniform distributions of valid moves, before self play.
- **Value Bootstrapping:** Boot strap the value head with a robust suite of perfect knowledge (mate evals, endgame tablebases, certain stockfish evaluations).

Chapter 7

Statement of Ethics

Bibliography

Bengio, Y. *et al.* (2009) "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. Montreal, Quebec, Canada: Association for Computing Machinery (ICML '09), pp. 41–48. Available at: https://doi.org/10.1145/1553374.1553380.

Huang, S. and Ontañón, S. (2020) "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," *CoRR* [Preprint]. Available at: https://arxiv.org/abs/2006.14171.

Jenner, E. *et al.* (2024) *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*. Available at: https://arxiv.org/abs/2406.00877.

McGrath, T. *et al.* (2022) "Acquisition of chess knowledge in AlphaZero," *Proceedings of the National Academy of Sciences*, 119(47). Available at: https://doi.org/10.1073/pnas.2206625119.

Monroe, D. and Chalmers, P.A. (2024) *Mastering Chess with a Transformer Model*. Available at: https://arxiv.org/abs/2409.12272.

Power, A. *et al.* (2022) "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177* [Preprint].

Schrittwieser, J. *et al.* (2020) "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 588(7839), pp. 604–609. Available at: https://doi.org/10.1038/s41586-020-03051-4.

Shannon, C.E. (1950) "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Sharma, S. *et al.* (2017) "Learning to Factor Policies and Action-Value Functions: Factored Action Space Representations for Deep Reinforcement learning," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1705.07269.

Silver, D. *et al.* (2017) *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Available at: https://arxiv.org/abs/1712.01815.

World Chess Federation (FIDE) (2023) "FIDE Laws of Chess". Available at: https://handbook.fide.com/chapter/E012023.